

BANKERS ALGORITHM FOR DEADLOCK AVOIDANCE

```
#include <stdio.h>
```

```
void inputMatrices(int n, int m, int alloc[n][m], int max[n][m], int avail[m]) {
```

```
    printf("Enter Allocation Matrix (%d x %d):\n", n, m);
```

```
    for (int i = 0; i < n; i++)
```

```
        for (int j = 0; j < m; j++)
```

```
            scanf("%d", &alloc[i][j]);
```

```
    printf("Enter Max Matrix (%d x %d):\n", n, m);
```

```
    for (int i = 0; i < n; i++)
```

```
        for (int j = 0; j < m; j++)
```

```
            scanf("%d", &max[i][j]);
```

```
    printf("Enter Available Resources (%d values):\n", m);
```

```
    for (int i = 0; i < m; i++)
```

```
        scanf("%d", &avail[i]);
```

```
}
```

```
void calculateNeed(int n, int m, int alloc[n][m], int max[n][m], int need[n][m]) {
```

```
    for (int i = 0; i < n; i++)
```

```
        for (int j = 0; j < m; j++)
```

```
            need[i][j] = max[i][j] - alloc[i][j];
```

```
}
```

```
void bankersAlgorithm(int n, int m, int alloc[n][m], int need[n][m], int avail[m]) {
```

```
    int f[n], ans[n], ind = 0;
```

```
    for (int i = 0; i < n; i++) f[i] = 0;
```

```
    for (int k = 0; k < n; k++) {
```

```

for (int i = 0; i < n; i++) {
    if (f[i] == 0) {
        int flag = 0;
        for (int j = 0; j < m; j++) {
            if (need[i][j] > avail[j]) {
                flag = 1;
                break;
            }
        }
        if (flag == 0) {
            ans[ind++] = i;
            for (int y = 0; y < m; y++) avail[y] += alloc[i][y];
            f[i] = 1;
        }
    }
}
}

```

```

printf("Safe Sequence: ");
for (int i = 0; i < n - 1; i++) printf("P%d -> ", ans[i]);
printf("P%d\n", ans[n - 1]);
}

```

```

int main() {
    int n, m;
    printf("Enter number of processes and resources:\n");
    scanf("%d %d", &n, &m);

    int alloc[n][m], max[n][m], avail[m], need[n][m];
    inputMatrices(n, m, alloc, max, avail);
    calculateNeed(n, m, alloc, max, need);
}

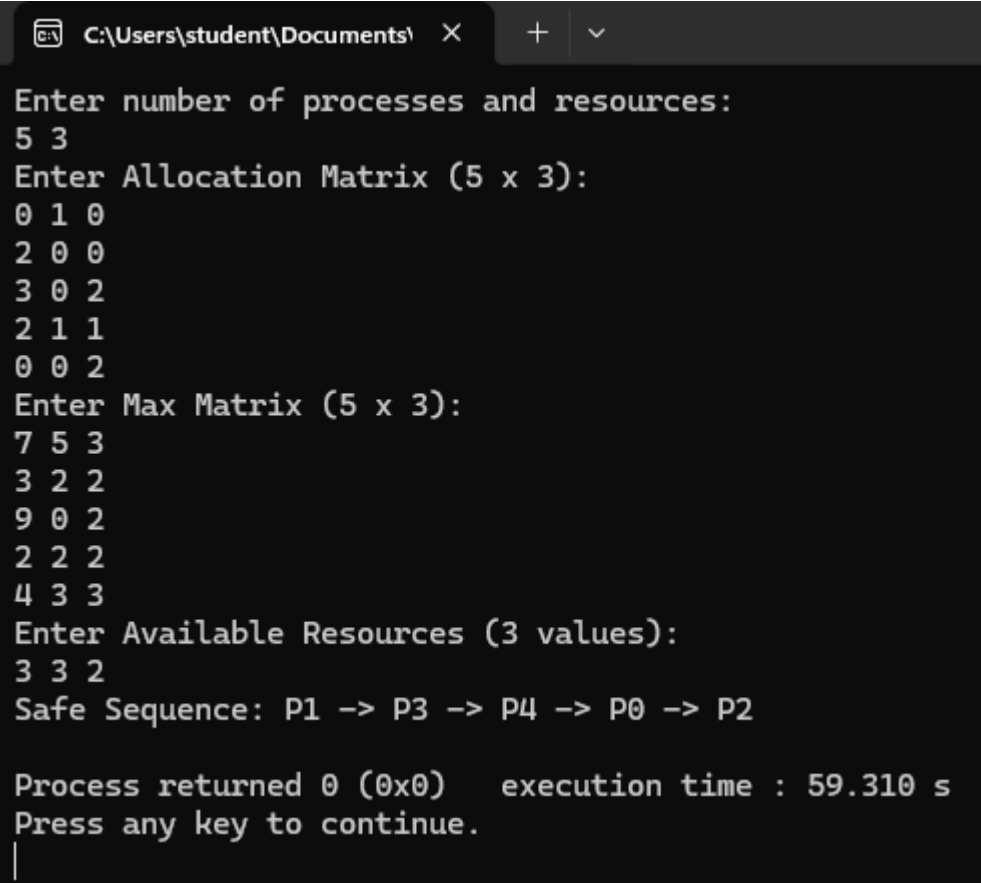
```

```
bankersAlgorithm(n, m, alloc, need, avail);
```

```
return 0;
```

```
}
```

OUTPUT:



```
C:\Users\student\Documents' X + v
Enter number of processes and resources:
5 3
Enter Allocation Matrix (5 x 3):
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Max Matrix (5 x 3):
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources (3 values):
3 3 2
Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2

Process returned 0 (0x0)    execution time : 59.310 s
Press any key to continue.
|
```

OBSERVATION:

```

    ans[ind++] = 1;
    for (int y = 0; y < m; y++)
        avail[y] += alloc[i][y];
    f[i] = 1;
}

printf("Safe Sequence: ");
for (int i = 0; i < n; i++)
    printf("P%d → ", ans[i]);
printf("P%d\n", ans[n-1]);

int main() {
    int n, m;
    printf("Enter number of process and resources: ");
    scanf("%d %d", &n, &m);

    int alloc[n][m], max[m][m], avail[m], need[n][m];
    inputMatrix(n, m, alloc, max, avail);
    calculateNeed(n, m, alloc, max, need);
    bankersAlgorithm(n, m, alloc, need, avail);

    return 0;
}

Output: Enter number of processes & resources: 5 3
Enter Allocation matrix (5x3):
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter max matrix (5x3):
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter Available resources (3 values): 3 3 2

Safe Sequence: P1 → P3 → P4 → P0 → P2

```

R
11/5/25

```

11/5/25 Banker's Algorithm for Deadlock Avoidance

#include <stdio.h>

void inputMatrix (int n, int m, int alloc[n][m],
                  int max[n][m], int avail[m]) {
    printf("Enter allocation matrix (%d x %d): \n", n, m);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);
}

void printMatrix (int n, int m, int max[n][m],
                  int alloc[n][m]) {
    printf("Enter max matrix (%d x %d): \n", n, m);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            scanf("%d", &max[i][j]);
}

void printMatrix (int n, int m, int avail[m]) {
    printf("Enter available matrix (%d): \n", m);
    for (int i = 0; i < m; i++)
        scanf("%d", &avail[i]);
}

void calculateNeed (int n, int m, int alloc[n][m],
                    int max[n][m], int need[n][m]) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            need[i][j] = max[i][j] - alloc[i][j];
}

void bankersAlgorithm (int n, int m, int alloc[n][m],
                       int need[n][m], int avail[m]) {
    int f[n], ans[n], ind = 0;
    for (int i = 0; i < n; i++) f[i] = 0;
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            if (f[i] == 0) {
                int flag = 0;
                for (int j = 0; j < m; j++)
                    if (need[i][j] > avail[j])
                        flag = 1;
                if (!flag)
                    break;
                if (flag == 0)

```